Experiment Report

(Experiment 1: Introduction to MATLAB and the Frequency Perspective of Probability)

Class:

Name:

Student ID:

Course Name: 概率和数理统计

Instructor:

Date: 2026.5.12

Experiment 1: Introduction to MATLAB and the Frequency Perspective of Probability

I、 Experiment Objectives

1. Understand the basic interface and programming logic of MATLAB.
2. Learn to generate random numbers and visualize data using basic plotting functions.
3. Intuitively understand the Law of Large Numbers (LLN) by observing how empirical frequencies converge to theoretical probabilities.
4. Comprehend the counter-intuitive nature of Conditional Probability and Bayes' Theorem by simulating real-world scenarios (e.g., the Monty Hall problem and medical testing paradoxes).

II、 Experiment Preview

Before attending the lab, students should review and familiarize themselves with the following key concepts:

1. **MATLAB Basics:** Basic syntax for variable definitions, matrix/array operations, logical indexing, control flow (loops and conditional statements), and standard plotting functions (e.g., `plot`, `histogram`).
2. **The Law of Large Numbers (LLN):** The frequentist interpretation of probability and how empirical relative frequencies converge to theoretical probabilities as the number of trials approaches infinity:
3. **Conditional Probability:** The mathematical definition of conditional probability and how initial probabilities change when new information is introduced.
4. **System Reliability (Independent Events):** Understanding how probabilities of independent events combine via intersection (Series/AND logic) and union (Parallel/OR logic).
5. **Total Probability and Bayes' Theorem:** The formulation of the Total Probability Theorem and Bayes' Theorem, specifically understanding how to calculate posterior probabilities from prior probabilities and likelihoods (e.g., understanding false positives in testing scenarios).

III、 Experiment Content

1. MATLAB Tutorial: Learn basic syntax, random number generation, array operations, and data visualization.
2. **Task 1 (Law of Large Numbers):** Simulate a coin toss experiment to visually verify how empirical frequencies converge to theoretical probabilities..
3. **Task 2 (Conditional Probability):** Simulate the Monty Hall Problem to compare the winning probabilities of the "Stay" versus "Switch" strategies.
4. **Task 3 (System Reliability):** Simulate a multi-component system using Monte Carlo methods to verify the rules of intersection (AND/Series) and union (OR/Parallel) for independent events.
5. **Task 4 (Bayes' Theorem):** Simulate a medical test to calculate true predictive values

IV、 Experiment Result

1. Task 1 (Law of Large Numbers)

    1) Source code:

```
clear; clc; close all;
N = 10000;
tosses = randi([0, 1], 1, N);
cum_heads = cumsum(tosses);
trials = 1:N;
freq = cum_heads ./ trials;
plot(trials, freq, 'b-', 'LineWidth', 1.5);
hold on;
yline(0.5, 'r--', 'LineWidth', 2);
title('Coin Toss and LLN');
xlabel('trials');
ylabel('cumulative frequency');
legend('empirical frequency', 'theoretical probability(0.5)');
grid on;
```

    2) Descriptions of the code:

        ① Generate 10000 coin tosses.

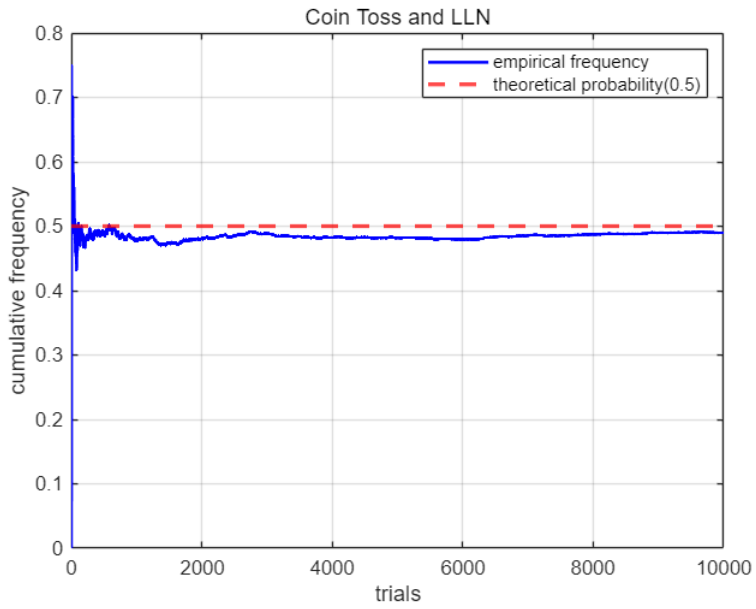
```
N = 10000;
tosses = randi([0, 1], 1, N);
```

        ② Calculate the cumulative sum of heads and the cumulative frequency.

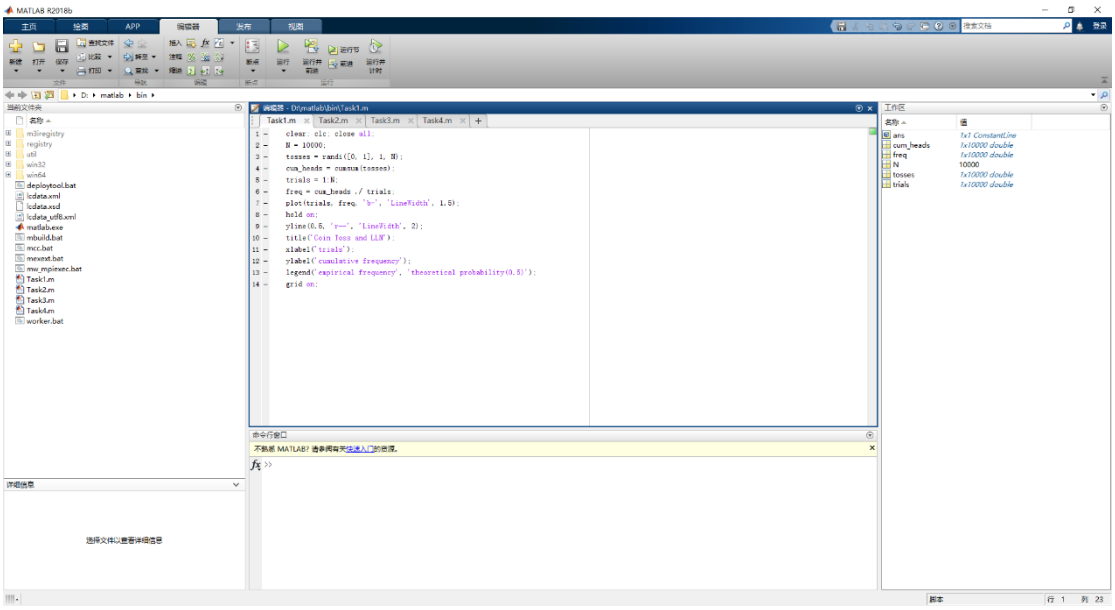
```
cum_heads = cumsum(tosses);
trials = 1:N;
freq = cum_heads ./ trials;
```

③ Plot trails (X) vs freq (Y). Add a red line at y = 0.5.

```
plot(trials, freq, 'b-', 'LineWidth', 1.5);
hold on;
yline(0.5, 'r--', 'LineWidth', 2);
title('Coin Toss and LLN');
xlabel('trials');
ylabel('cumulative frequency');
legend('empirical frequency', 'theoretical probability(0.5)');
grid on;
```



3) Screenshot of the terminal output:



2. Task 2 (Conditional Probability)

1) Source code:

```
clear; clc;
N = 10000;
cars = randi([1, 3], 1, N);
choices = randi([1, 3], 1, N);
stay_wins = sum(choices == cars);
switch_wins = sum(choices ~= cars);
stay_rate = stay_wins / N;
switch_rate = switch_wins / N;
disp('Monty Hall');
fprintf('Win rate if you stay: %.4f (%.2f%%)\n', stay_rate, stay_rate *
100);
fprintf('Win rate if you switch: %4f (%.2f%%)\n', switch_rate, switch_rate
* 100);
```

2) Descriptions of the code:

① Randomly place cars behind Door 1, 2, or 3. Randomly pick initial doors for the player.

```
cars = randi([1, 3], 1, N);
choices = randi([1, 3], 1, N);
```

② Count how many times “stay” wins and “switch” wins respectively. Calculate their rate.

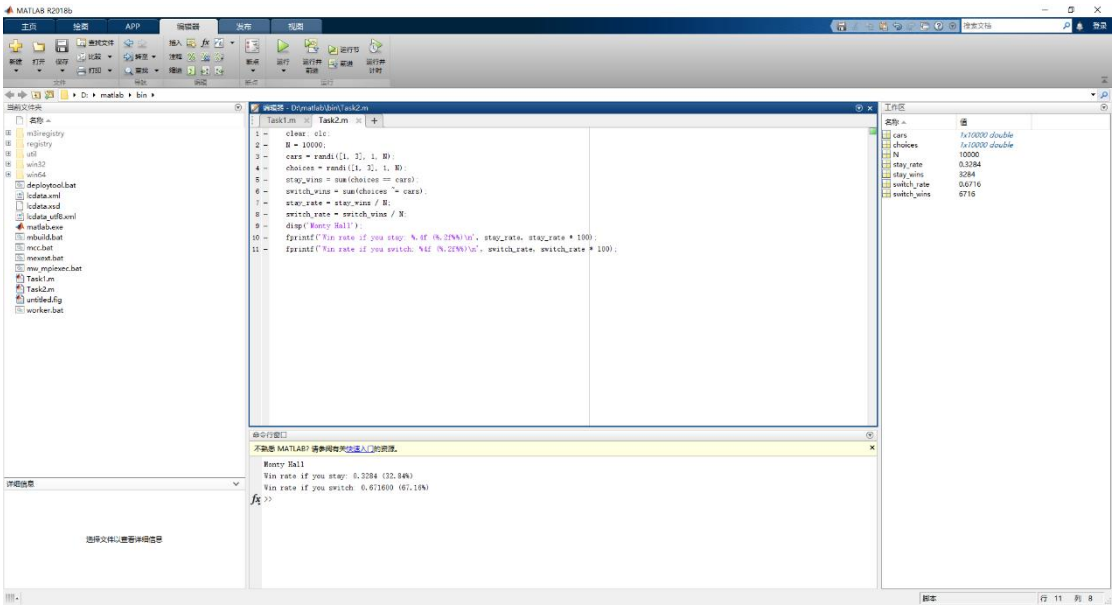
```
stay_wins = sum(choices == cars);
switch_wins = sum(choices ~= cars);
```

```
stay_rate = stay_wins / N;
switch_rate = switch_wins / N;
```

③ Print the results.

```
disp('Monty Hall');
fprintf('Win rate if you stay: %.4f (%.2f%%)\n', stay_rate, stay_rate *
100);
fprintf('Win rate if you switch: %4f (%.2f%%)\n', switch_rate, switch_rate
* 100);
```

3) Screenshot of the terminal output:



3. Task 3 (System Reliability)

1) Source code:

```
clear; clc;
N = 10000;
A = rand(1, N) < 0.9;
B = rand(1, N) < 0.8;
C = rand(1, N) < 0.85;
D = rand(1, N) < 0.95;
E = rand(1, N) < 0.7;
System = (A & B) | (C & D) | E;
simulated_reliability = mean(System);
%Theoretical
P_Branch1 = 0.9 * 0.8;
P_Branch2 = 0.85 * 0.95;
P_Branch3 = 0.7;
theoretical_reliability = 1 - ((1 - P_Branch1) * (1 - P_Branch2) * (1 -
P_Branch3));
disp('Simulated Reliability and Theoretical Reliability');
```

```
 fprintf('Simulated Reliability: %.5f\n', simulated_reliability);
 fprintf('Theoretical Reliability: %.5f\n', theoretical_reliability);
 fprintf('Absolute Difference: %.5f\n', abs(simulated_reliability -
theoretical_reliability));
```

2) Descriptions of the code:

① Generate status arrays for components.

```
 A = rand(1, N) < 0.9;
 B = rand(1, N) < 0.8;
 C = rand(1, N) < 0.85;
 D = rand(1, N) < 0.95;
 E = rand(1, N) < 0.7;
```

② Combine logic: System = (A & B) | (C & D) | E, and calculate simulated reliability.

```
 System = (A & B) | (C & D) | E;
 simulated_reliability = mean(System);
```

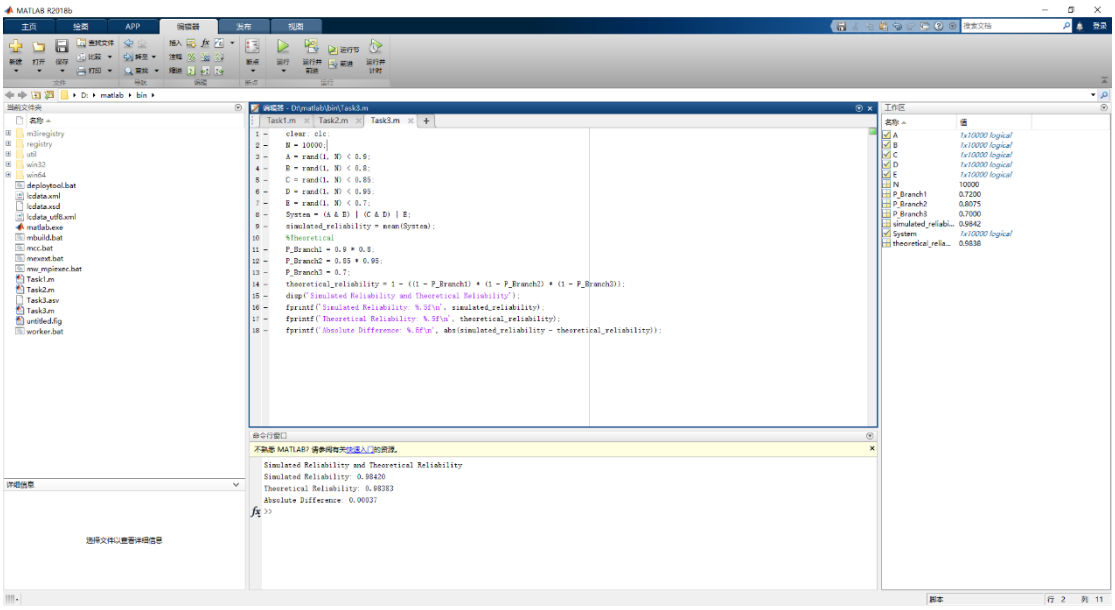
③ Calculate theoretical reliability using math formulas.

```
 P_Branch1 = 0.9 * 0.8;
 P_Branch2 = 0.85 * 0.95;
 P_Branch3 = 0.7;
 theoretical_reliability = 1 - ((1 - P_Branch1) * (1 - P_Branch2) * (1 -
P_Branch3));
```

④ Print them and their difference.

```
 disp('Simulated Reliability and Theoretical Reliability');
 fprintf('Simulated Reliability: %.5f\n', simulated_reliability);
 fprintf('Theoretical Reliability: %.5f\n', theoretical_reliability);
 fprintf('Absolute Difference: %.5f\n', abs(simulated_reliability -
theoretical_reliability));
```

3) Screenshot of the terminal output:



4. Task 4 (Bayes' Theorem)

1) Source code:

```
clear; clc;
N = 1000000;
has_disease = rand(N, 1) < 0.01;
test_result = false(N, 1);
test_result(has_disease) = rand(sum(has_disease), 1) < 0.95;
test_result(~has_disease) = rand(sum(~has_disease), 1) < 0.05;
total_pos = sum(test_result);
true_pos = sum(test_result & has_disease);
P_simulated = true_pos / total_pos;
%Theoretical
P_d = 0.01;
P_t_on_d = 0.95;
P_t_on_nd = 0.05;
P_d_on_t = (P_t_on_d * P_d) / ((P_t_on_d * P_d) + (P_t_on_nd * (1 - P_d)));
disp('Simulated and Theoretical');
fprintf('Simulated P(D|T): %.5f\n', P_simulated);
fprintf('Theoretical P(D|T): %.5f\n', P_d_on_t);
```

2) Descriptions of the code:

① Generate sick/healthy people.

```
has_disease = rand(N, 1) < 0.01;
```

② Simulate test results, then test the sick and the healthy respectively.

```
test_result = false(N, 1);
test_result(has_disease) = rand(sum(has_disease), 1) < 0.95;
```

```
test_result(~has_disease) = rand(sum(~has_disease), 1) < 0.05;
```

③ Find total positive tests and TRUE positives (sick and tested positive). Calculate Empirical P(D|T)

```
total_pos = sum(test_result);
true_pos = sum(test_result & has_disease);
P_simulated = true_pos / total_pos;
```

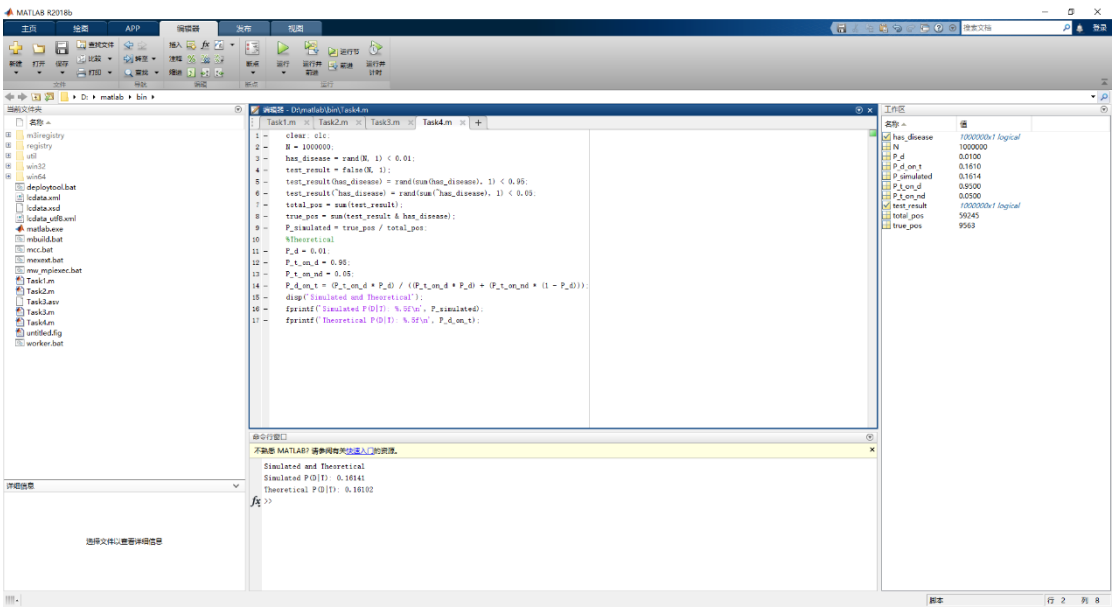
④ Calculate Theoretical P(D|T) using the Bayes' Theorem math formula.

```
P_d = 0.01;
P_t_on_d = 0.95;
P_t_on_nd = 0.05;
P_d_on_t = (P_t_on_d * P_d) / ((P_t_on_d * P_d) + (P_t_on_nd * (1 - P_d)));
```

⑤ Print the results.

```
disp('Simulated and Theoretical');
fprintf('Simulated P(D|T): %.5f\n', P_simulated);
fprintf('Theoretical P(D|T): %.5f\n', P_d_on_t);
```

3) Screenshot of the terminal output:



5. Task 5 (Base Rate Sensitivity Curve)

1) Source code:

```
clear; clc; close all;
P_d = 0:0.01:1;
P_t_on_d = 0.95;
P_t_on_nd = 0.05;
```

```
P_d_on_t = (P_t_on_d * P_d) ./ ((P_t_on_d * P_d) + (P_t_on_nd * (1 - P_d)));
plot(P_d, P_d_on_t, 'Color', [211/255, 150/255, 249/255], 'LineStyle',
'-', 'LineWidth', 2);
title('Base rate sensitivity curve');
xlabel('Probability of having the disease');
ylabel('Posterior probability');
grid on;
```

2) Descriptions of the code:

① Set the range of the probability of having the disease from 0 to 1.

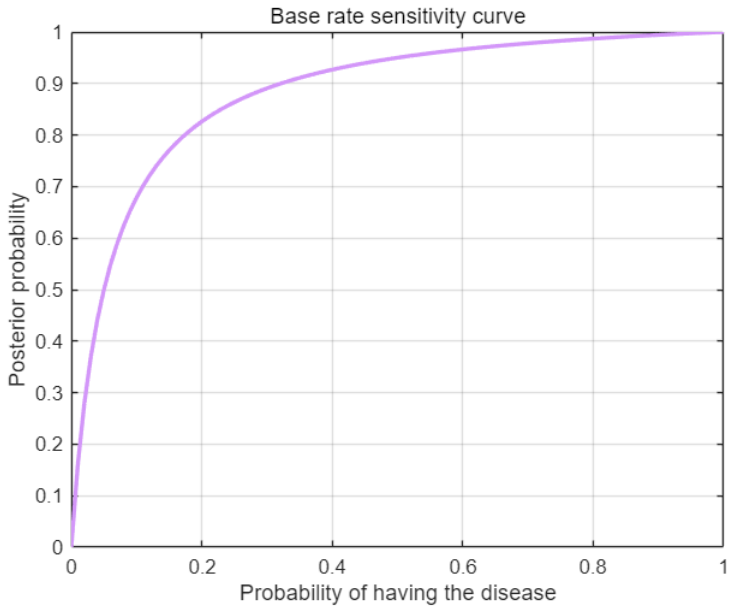
```
P_d = 0:0.01:1;
```

② Calculate the Posterior probability using the Bayes' Theorem math formula.

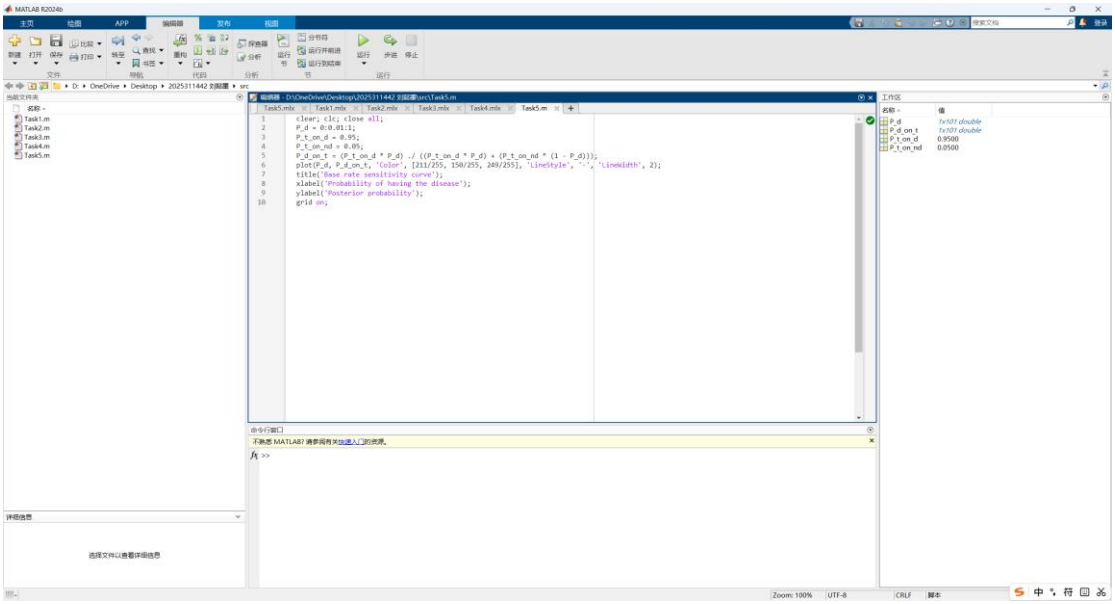
```
P_t_on_d = 0.95;
P_t_on_nd = 0.05;
P_d_on_t = (P_t_on_d * P_d) ./ ((P_t_on_d * P_d) + (P_t_on_nd * (1 - P_d)));
```

③ Plot the base rate sensitivity curve.

```
plot(P_d, P_d_on_t, 'Color', [211/255, 150/255, 249/255], 'LineStyle',
'-', 'LineWidth', 2);
title('Base rate sensitivity curve');
xlabel('Probability of having the disease');
ylabel('Posterior probability');
grid on;
```



3) Screenshot of the terminal output:



V、 Extended Questions

- **Task 1 Results:** At approximately what toss number n did the line stop wildly fluctuating and stabilize?

  From the coin toss graph, the cumulative frequency fluctuated very strongly at the beginning. After about n = 1000 to 2000 tosses, the line stopped wildly fluctuating and began to stabilize near the theoretical probability 0.5. Although there were still small fluctuations later, the overall trend became much smoother and closer to 0.5 as the number of trials increased.

- **Task 2 Results:** Write down your simulated win rates. Do your results match the theory that the probability of winning the car is $1/3$ if you stay, and $2/3$ if you switch?

  The simulated win rates were:

  Win rate if staying: 0.3284, or 32.84%

  Win rate if switching: 0.6716, or 67.16%

These results match the theoretical probabilities very well. The probability of winning the car is about 1/3 if the player stays, and about 2/3 if the player switches. Therefore, the simulation supports the conclusion that switching is a better strategy in the Monty Hall problem.

- **Task 3 Results:** Report the simulated and theoretical reliability. Briefly explain why having parallel branches (OR logic) makes the whole system more reliable than any single component.

  The results were:

  Simulated reliability: 0.98420

  Theoretical reliability: 0.98383

  Absolute difference: 0.00037

  The simulated result is very close to the theoretical result, so the simulation is successful. Parallel branches use OR logic, which means the whole system can still work as long as at least one branch works. The system fails only when all branches fail at the same time. Since this probability is much smaller than the failure probability of a single component, the whole system becomes more reliable than any single component.

- **Task 4 Results:** Report the simulated and theoretical $P(D|T)$. Based on your numbers, explain to a patient why a positive result on a 95% accurate test does not mean they have a 95% chance of dying.

  The results were:

  Simulated P(D|T): 0.16141

  Theoretical P(D|T): 0.16102

The simulated value is very close to the theoretical value, so the simulation agrees with Bayes' theorem.

Although the test has 95% accuracy, a positive result does not mean the patient has a 95% chance of having the disease or dying. This is because the disease itself is rare, with only a 1% prior probability. Among a large number of healthy people, even a small false positive rate of 5% can still produce many positive test results. Therefore, after receiving a positive result, the actual probability is only about 16.1%, not 95%.

VI、 Summary and Experiment Reflections

Gains: I learned how to use MATLAB to write easy constructions, plot curves, generate arrays, output text, combine logic and simulate test results.

Challenges: To write this experiment report, I learned how to beautify the MATLAB codes in Microsoft Word. According to the Internet, I saved my .m files as .mlx files, then export the .mlx file to Word. It did work, however, when I tried to run my initial .m files again, I found out the error like this.

Even though I tried as it read, deleting the .mlx file (I even deleted it from the bin) and rerun the .m file, this error still existed. So I wonder how to insert MATLAB source code into Microsoft Word elegantly.